

初中学生编程入门规划

一、规划定位

本规划面向已经开始学习或即将学习计算机基础知识的初中学生。目标不是让学生一开始就背大量语法，也不是快速追求“写很多代码”，而是让学生逐步形成：会使用工具、能理解问题、能写出简单程序、能调试错误、能完成小项目的能力。

建议总体路线：

- 计算机基础
 - 编程前准备
 - 图形化编程入门
 - Python 基础编程
 - 网页或硬件方向拓展
 - 综合小项目

学生学完后应能做到：

1. 知道程序是由明确指令组成的，计算机只会按规则执行。
2. 能用“输入、处理、输出”的方式描述一个程序。
3. 能理解并使用变量、表达式、顺序结构、分支结构、循环结构和函数。
4. 能用自然语言、流程图或伪代码先设计思路，再写代码。
5. 能独立管理代码文件，保存、查找、压缩并提交项目。
6. 能阅读简单错误提示，逐步定位并修改常见错误。
7. 能完成一个有明确主题的小程序或小网页项目。
8. 能负责任地使用网络资料和 AI 工具，不直接复制、不随意发布隐私信息。

二、学习编程前的准备要求

正式进入编程前，学生最好先具备以下基础能力。这些能力来自计算机基础知识学习，是后续编程能否顺利进行的关键。

准备能力	达标表现	为什么重要
文件管理	能新建项目文件夹，保存、查找、复制、压缩文件	代码、图片、数据文件都需要按项目管理
路径理解	知道文件在哪个文件夹，能理解相对路径和绝对路径的基本含义	避免程序找不到图片、数据或代码文件
英文输入	能输入英文括号、引号、冒号、逗号、等号、缩进等	编程语言对符号非常严格

快捷键使用	会复制、粘贴、撤销、保存、查找、切换窗口	提高编程和调试效率
数据基础	知道计算机用数字表示文字、图片、声音和程序	帮助理解变量、数据类型和文件大小
网络基础	知道浏览器、网址、服务器、上传、下载的含义	帮助理解网页、在线编程平台和资源引用
计算思维	能把任务拆成步骤，能用流程图描述判断和重复	编程前先想清楚步骤，代码才容易写
调试意识	出错后能看提示、查位置、改一处、再运行	程序出错是常态，关键是要排查

如果学生还不能熟练管理文件、输入英文符号或理解流程图，建议先补充 2 到 4 课时的编程前准备课。

三、编程语言选择建议

初中阶段不建议一开始同时学习太多语言。更适合采用“一条主线，少量拓展”的方式。

1. 推荐主线

阶段	推荐工具或语言	适合原因	学习重点
入门阶段	Scratch、Blockly、Kitten、Code.org 等图形化编程	减少语法错误，让学生先理解程序结构	顺序、事件、分支、循环、变量、角色交互
基础阶段	Python	语法相对简洁，适合数学、数据、小游戏和自动化任务	输入输出、变量、条件、循环、列表、函数、调试
拓展阶段	HTML、CSS、JavaScript	成果可视化强，适合制作网页和交互作品	网页结构、样式、按钮交互、表单、浏览器运行
兴趣方向	MakeCode、Arduino、micro:bit、机器人平台	能把程序和现实设备连接起来	传感器、控制、事件、循环、硬件安全

2. 不建议的做法

1. 不建议一开始就学习 C/C++、Java 等语法负担较重的语言，除非目标是信息学竞赛并有充足课时支持。
2. 不建议频繁更换语言。学生刚理解变量和循环时，频繁换语法会增加负担。
3. 不建议只让学生复制代码运行。每个程序都应先说清“输入是什么、处理什么、输出什么”。
4. 不建议用“能运行”作为唯一评价标准，还要看文件组织、命名、思路说明和调试记录。

四、总体学习阶段

建议分为五个阶段。课时可根据学校安排压缩或扩展。

阶段	建议课时	核心目标	主要成果
第 0 阶段：编程前准备	2 课时	熟悉工具、文件夹、英文符号和流程表达	建好个人编程文件夹，完成一个流程图
第 1 阶段：图形化编程	6 课时	用积木理解程序结构	完成一个动画、游戏或互动故事
第 2 阶段：Python 基础	12 课时	掌握基础语法和解决问题方法	完成 3 到 5 个小程序
第 3 阶段：网页或硬件拓展	6 课时	体验不同编程应用方向	完成一个个人网页或硬件交互作品
第 4 阶段：综合项目	4 课时	独立设计、实现、测试和展示	完成一个完整小项目并展示

完整方案约 30 课时。若课时有限，可采用 16 课时最小方案：

编程前准备 2 课时
图形化编程 4 课时
Python 基础 8 课时
综合项目 2 课时

五、详细课程规划

第 0 阶段：编程前准备

第 1 课时：认识编程和开发工具

学习内容：

1. 什么是程序：程序是一组让计算机执行的明确指令。
2. 什么是编程：把解决问题的步骤写成计算机能执行的形式。
3. 认识编程工具：浏览器、图形化编程平台、代码编辑器、运行窗口。
4. 建立个人编程文件夹，如 信息科技学习/程序代码/。
5. 学会保存、打开、重命名和压缩项目文件。

课堂任务：

1. 新建 编程入门 文件夹。
2. 在其中建立 图形化编程、Python练习、网页作品、项目资料 四个子文件夹。
3. 新建一个 学习记录.txt，写下“我想用编程解决的一个问题”。

达标要求：

- 能找到自己的项目文件夹。
- 能说明“代码文件不能随便移动，否则可能找不到资源”。
- 能保存并重新打开自己的练习文件。

第 2 课时：用流程图表达程序思路

学习内容：

1. 输入、处理、输出模型。
2. 顺序结构：一步接一步执行。
3. 分支结构：如果满足条件就执行某一步。
4. 循环结构：重复执行一组步骤。
5. 用自然语言和流程图描述算法。

课堂任务：

画出“判断成绩是否及格”的流程：

```
graph TD
    A[输入成绩] --> B{如果成绩大于或等于 60}
    B -- 是 --> C[输出“及格”]
    B -- 否 --> D[输出“不及格”]
```

达标要求：

- 能说出程序的输入、处理和输出。
- 能把一个生活任务拆成 3 到 5 个步骤。
- 能读懂简单的“如果……否则……”流程。

第 1 阶段：图形化编程入门

图形化编程的重点不是“玩积木”，而是先让学生理解程序结构，减少英文符号和语法错误带来的挫败感。

课时	主题	核心概念	课堂任务
第 3 课时	第一个动画	事件、顺序执行、角色、舞台	点击绿旗后角色说话并移动
第 4 课时	角色和坐标	坐标、方向、速度、外观	制作角色移动动画
第 5 课时	判断和选择	条件、如果、否则	制作“碰到边缘就反弹”或“答题判断”
第 6 课时	重复执行	循环、次数、条件	制作重复画图或自动移动效果
第 7 课时	变量和计分	变量、赋值、更新	制作一个简单计分游戏
第 8 课时	图形化综合作品	事件、分支、循环、变量组合	完成互动故事、迷宫或问答小游戏

图形化阶段重点

学生应理解：

- 1. 程序从某个事件开始运行，比如点击绿旗、按下按键、点击角色。
- 2. 积木从上到下执行，顺序不同，结果可能不同。
- 3. 条件判断让程序根据情况做不同事情。
- 4. 循环让程序重复做事，避免写很多重复指令。
- 5. 变量可以记录分数、时间、生命值、姓名等数据。

推荐作品

作品	练习内容
自我介绍动画	事件、顺序、角色外观、文字输出
迷宫小游戏	坐标、键盘控制、条件判断
问答小程序	输入、判断、变量计分
接苹果游戏	循环、随机位置、碰撞检测、分数
校园文明宣传动画	场景切换、角色对话、事件控制

阶段达标标准

完成图形化编程阶段后，学生应能：

- 1. 独立搭建一个能运行的简单程序。
- 2. 解释程序从哪里开始执行。
- 3. 在作品中至少使用一次条件判断和一次循环。
- 4. 使用变量记录一个变化的数据。
- 5. 修改程序参数并观察结果变化。

第 2 阶段：Python 基础编程

Python 阶段是编程入门的主线。教学重点应放在“理解概念”和“解决问题”，不要让学生机械背语法。

课时	主题	学习内容	小任务
第 9 课时	第一个 Python 程序	编辑器、运行、print()、保存 .py 文件	输出自我介绍
第 10 课时	输入和输出	input()、字符串、程序交互	输入姓名，输出问候语
第 11 课时	变量和数据	变量命名、字符串、整数、小数	计算购物总价

第 12 课时	表达式和运算	加减乘除、取余、比较运算	判断一个数是奇数还是偶数
第 13 课时	条件判断	if、elif、else、缩进	成绩等级判断
第 14 课时	循环一：固定次数	for、range()、计数	输出 1 到 100，计算累加和
第 15 课时	循环二：满足条件	while、循环条件、避免死循环	猜数字游戏
第 16 课时	字符串处理	长度、拼接、查找、大小写	制作简单口令检查
第 17 课时	列表	多个数据、下标、遍历、添加删除	记录班级兴趣爱好
第 18 课时	函数	定义函数、参数、返回值	封装平均分计算
第 19 课时	调试和测试	错误提示、语法错误、逻辑错误、测试数据	修改有错误的程序
第 20 课时	Python 小项目	综合使用变量、分支、循环、函数	完成一个命令行小程序

Python 阶段核心概念

概念	学生应达到的理解
输出	程序把结果显示给用户
输入	程序接收用户提供的数据
变量	给数据起名字，方便后面使用
数据类型	不同数据有不同用途，如文字、整数、小数
表达式	能计算出结果的式子
条件判断	根据条件选择不同操作
循环	重复执行一组操作
列表	用一个名字保存多个数据
函数	把一组操作封装起来，方便重复使用
调试	找到错误位置，理解原因并修正

Python 小程序示例

项目	涉及知识
个性化问候程序	输入、输出、字符串
购物金额计算器	变量、数字、表达式
成绩等级判断	条件判断、边界条件

奇偶数判断器	取余、比较、分支
猜数字游戏	随机数、循环、判断
单词记忆小助手	列表、循环、字符串
班级平均分计算	列表、累加、函数
简易问答计分系统	输入、条件、变量、循环

Python 阶段常见错误和训练方法

常见错误	训练方法
中文标点导致报错	每次运行前检查括号、引号、冒号是否为英文符号
缩进错误	强调同一层代码左边对齐，条件和循环内部必须缩进
变量名拼写不一致	让学生用有意义的英文变量名，并保持前后一致
输入数字当成文字	讲清 input() 得到的是文本，需要转换成数字
循环停不下来	先用自然语言写循环条件，再写代码
结果能运行但不正确	用简单数据、边界数据和特殊数据测试

Python 阶段达标标准

完成 Python 基础后，学生应能：

1. 独立创建、保存并运行 .py 文件。
2. 写出包含输入、处理和输出的简单程序。
3. 正确使用变量保存数据。
4. 使用条件判断解决分类问题。
5. 使用循环处理重复任务。
6. 能读懂并修改简单函数。
7. 能根据错误提示定位常见语法错误。
8. 能用不同测试数据检查程序结果。

第 3 阶段：网页或硬件方向拓展

这一阶段可根据学校条件和学生兴趣选择一个方向，不必全部学习。

方向 A：网页制作入门

适合希望看到直观作品、对设计和展示感兴趣的学生。

课时	主题	学习内容	小任务
第 21 课时	第一个网页	HTML、标签、标题、段落	制作个人介绍网页
第 22 课时	图片和链接	图片路径、超链接、文件组织	制作校园风景页面

第 23 课时	页面样式	CSS、颜色、字体、边距	美化个人网页
第 24 课时	页面布局	盒子、列表、简单排版	制作兴趣展示页
第 25 课时	简单交互	JavaScript、按钮、事件	点击按钮切换文字或颜色
第 26 课时	网页小作品	HTML、CSS、JavaScript 综合	制作主题展示网站

网页方向重点：

1. HTML 负责网页结构。
2. CSS 负责网页样式。
3. JavaScript 负责网页交互。
4. 图片路径和文件夹组织非常重要。
5. 网页作品发布前要注意版权、隐私和内容规范。

推荐主题：

- 我的兴趣介绍页
- 校园文明宣传页
- 信息安全知识宣传页
- 班级活动展示页
- 科学小知识互动页

方向 B：硬件或机器人编程入门

适合有 micro:bit、Arduino、机器人套件或创客设备的学校。

主题	学习内容	小任务
灯光控制	输出、循环、延时	制作闪烁灯
按钮输入	输入、事件、条件	按下按钮显示不同图案
传感器	数据读取、阈值判断	光线暗时自动亮灯
声音和显示	输出设备、反馈	制作倒计时提醒器
综合控制	输入、处理、输出	制作简易安全提示装置

硬件方向重点：

1. 程序可以控制现实设备。
2. 传感器相当于程序的输入。
3. 灯、屏幕、声音、马达相当于程序的输出。
4. 硬件连接要注意安全，不随意短接、拆接或使用不明电源。

第 4 阶段：综合项目

综合项目是编程入门学习的收束。重点是让学生经历一次完整过程：选题、分析、设计、实现、测试、展示。

项目流程

步骤	学生要做什么	产出
选题	选择一个自己能完成的小问题	项目题目
需求分析	说明程序要解决什么问题，给谁使用	需求说明
设计算法	写出输入、处理、输出和流程	流程图或伪代码
编写程序	分步骤实现功能	代码文件
测试修改	用不同数据测试，记录错误和修改	测试记录
整理提交	规范命名、压缩项目、准备展示	项目文件夹
展示反思	说明作品功能、亮点和不足	展示文稿或口头汇报

推荐综合项目

项目	难度	适合语言	核心知识
成绩等级判断器	入门 Python		输入、数字转换、条件判断
猜数字游戏	入门 Python		随机数、循环、分支
简易计算器	入门 Python 或网页		输入、表达式、条件
单词记忆小助手	基础 Python		列表、循环、字符串
问答计分系统	基础 Python 或 Scratch		变量、分支、循环
校园安全宣传网页	基础 HTML/CSS/JavaScript		网页结构、样式、交互
班级兴趣统计	提高 Python		列表、统计、输出
micro:bit 倒计时提醒器	提高 MakeCode 或 Python		输入、输出、事件、循环

项目要求

一个合格的入门项目应包含：

1. 一个清楚的项目文件夹。
2. 一个主程序文件或网页入口文件。
3. 一份简单说明，写明项目功能、运行方法和使用的数据。
4. 至少 3 组测试数据。
5. 至少 1 条调试记录，说明遇到的问题和解决过程。
6. 展示时能说明用了哪些编程概念。

六、学习方法建议

1. 每个程序都按五步学习

看懂问题

- 写出步骤
- 编写代码
- 运行测试
- 修改总结

不要直接跳到写代码。初学者最容易犯的错误是“还没想清楚，就开始敲代码”。

2. 用 IPO 模型理解程序

问题

示例

输入是什么？ 用户输入成绩

处理什么？ 判断成绩是否大于等于 60

输出什么？ 显示“及格”或“不及格”

示例：

任务：判断一个学生是否及格

输入：成绩 score

处理：如果 `score >= 60`，则及格；否则不及格

输出：判断结果

3. 先用自然语言，再写代码

例如学习循环时，先让学生说出：

从 1 开始

每次输出当前数字

数字加 1

如果数字不超过 10，就继续

否则结束

再过渡到代码。这样学生更容易理解循环变量和循环条件。

4. 建立调试习惯

每次出错时，按下面顺序检查：

1. 看错误提示，找到行号或关键字。
2. 检查刚刚修改过的地方。
3. 检查英文符号、大小写、缩进和文件名。
4. 用最简单的数据重新测试。
5. 一次只修改一个地方，运行后再观察结果。
6. 把错误原因和解决方法写进错题本。

5. 正确使用网络和 AI 工具

学生可以用搜索引擎、官方文档、教材资料和 AI 工具辅助学习，但要遵守规则：

1. 不直接复制大段代码当作自己的作品。
 2. 使用网上资料要记录来源。
 3. AI 给出的代码必须自己读懂、运行和测试。
 4. 不向 AI 或陌生网站输入真实姓名、电话、住址、账号密码等隐私信息。
 5. 遇到不理解的代码，要能用自己的话解释每一部分作用。
-

七、不同课时条件下的安排

1. 课时较少：16 课时方案

阶段	课时	内容
编程前准备	2	文件管理、工具使用、流程图、IPO 模型
图形化编程	4	事件、顺序、分支、循环、变量
Python 基础	8	输出、输入、变量、条件、循环、列表、函数、调试
综合项目	2	完成一个 Python 或图形化小项目

适合目标：让学生完成基础入门，能写简单程序。

2. 标准安排：30 课时方案

阶段	课时	内容
编程前准备	2	工具、文件夹、流程图、IPO 模型
图形化编程	6	动画、事件、变量、判断、循环、小游戏
Python 基础	12	输入输出、变量、条件、循环、字符串、列表、函数、调试
网页或硬件拓展	6	选择网页制作或硬件编程
综合项目	4	项目设计、实现、测试、展示

适合目标：系统完成编程入门，并形成项目经验。

3. 兴趣提升：40 课时以上方案

可在标准方案基础上增加：

1. Python 文件读写和简单数据统计。
2. Turtle 绘图或小游戏开发。
3. 更完整的 HTML/CSS/JavaScript 网页项目。
4. micro:bit、Arduino 或机器人控制。
5. 信息学竞赛基础：枚举、模拟、排序、简单搜索。

6. 更规范的项目展示、版本记录和作品迭代。

八、评价标准

评价不只看程序是否运行，还要看学生是否真正理解过程。

评价维度	达标表现
文件组织	项目文件夹清晰，文件命名规范，能正确提交
问题分析	能说清输入、处理和输出
程序实现	能使用变量、分支、循环等基础结构
调试能力	能根据错误提示修改常见问题
测试意识	能使用多组数据验证结果
表达能力	能解释代码思路，而不是只说“我复制的”
安全规范	不泄露隐私，尊重版权，合理使用 AI 和网络资料
项目完成度	作品功能明确，能演示，能说明不足和改进方向

建议评分示例：

项目	分值
项目文件和命名规范	10
思路说明或流程图	15
程序功能实现	35
测试和调试记录	20
展示表达和反思	15
安全、版权和引用规范	5

九、常见困难与教学对策

常见困难	原因	对策
不会保存或找不到文件	文件管理基础薄弱	每节课固定项目目录，提交前检查路径和文件名
经常输错符号	中英文标点混用	建立“英文符号检查表”，运行前先自查
看不懂报错	怕英文、怕错误	教学生先找行号和关键词，不要求一开始读懂全部英文
只会复制代码	没有先分析问题	每次写代码前必须写 IPO 或流程图
条件判断混乱	不清楚条件和结果	用生活例子训练“如果……否则……”
循环难理解	不知道重复的起点、条件和变化	先用表格模拟每次循环变量的变化

程序能运行但结果错	缺少测试意识	要求至少使用正常数据、边界数据、特殊数据测试
项目过大做不完	选题不清或功能太多	项目先做最小版本，再增加功能

十、最小必学清单

如果时间有限，初中学生编程入门至少应掌握以下内容：

1. 编程工具使用：能创建、保存、运行和提交代码文件。
2. 文件与路径：能管理项目文件夹，理解扩展名和资源路径。
3. 英文符号和缩进：能正确输入常见编程符号，知道缩进会影响程序。
4. IPO 模型：能说清程序的输入、处理和输出。
5. 变量和表达式：能用变量保存数据并进行简单计算。
6. 条件判断：能根据不同条件输出不同结果。
7. 循环结构：能处理重复任务。
8. 列表或多个数据：能保存并遍历一组数据。
9. 函数初步：能理解把重复操作封装起来。
10. 调试与测试：能看提示、查位置、改错误，并用多组数据验证。
11. 项目表达：能说明自己的程序解决了什么问题、如何运行、还有什么不足。
12. 数字安全：能保护隐私、尊重版权、谨慎使用网络和 AI 工具。

学完这些内容后，学生可以继续选择 Python 深入学习、网页制作、机器人编程、数据处理或信息学竞赛方向。最重要的是：学生要逐步从“照着做”转向“想清楚、写出来、测一测、改一改”。